endava

endava

Debugging & Unit Testing: Building Confidence in Code.

Alejandro Taborda Cadavid – Senior QA tester

Agenda

01 Why unit testing is not “extra work” but risk reduction

02 What is Debugging?

03 What is a Unit Test?

04 Connecting Debugging & Unit Testing

05 QA Perspective - Curiosities

Chapter

01 Why unit testing is not “extra work” but risk reduction

“As a QA Automation Engineer, my job is not just to find bugs — but to prevent them from reaching production. Debugging and unit testing are two of the most powerful tools we have for that.”

Chapter

02

What is Debugging?

- Process of identifying the root cause of a defect
- Not just fixing symptoms
- Requires systematic thinking

•Adding random logs everywhere

•Changing code without understanding the issue

•Assuming instead of verifying

•Ignoring edge cases

COMMON MISTAKES

Chapter

03 What is a Unit Test?

- Tests the smallest testable piece of code (usually a function).
- Isolated.
- Fast.
- Deterministic.

```python
def test_divide_normal_case():
    # Arrange
    a = 10
    b = 2

    # Act
    result = divide(a, b)

    # Assert
    assert result == 5
```

- Arrange ->Prepare data.
- Act -> Call the function.
- Assert -> Verify expected result.
- Make sure you test Edge cases

Structure of a Unit Test (AAA Pattern)

Chapter

03

Connecting Debugging & Unit Testing

- Debugging fixes today's bug
- Unit tests prevent tomorrow's bug
- Every bug found → should lead to a new unit test

"If you fix a bug without writing a test for it, you're accepting that it might come back."

Chapter

04

QA Perspective - Curiosities

```
Frontend debugging: DevTools Console + Network + Sources breakpoints, React/Redux devtools, performance profiling.

Backend debugging: Repro request + logs, IDE debugger, tracing/metrics.
```

```
Frontend unit tests: Vitest/Jest + Testing Library

Backend unit tests: Language runner + mock library + HTTP stubs + Mocha
```

Chapter

05

Unit Test Example

```
## CODE
export function applyDiscount(price, discountPercent) {
  return price - (price * discountPercent) / 100;
}

## UNIT TEST
test("treats string numbers as valid inputs", () => {
  expect(applyDiscount("100", "10")).toBe(90);
});

## RESOLUTION
export function applyDiscount(price, discountPercent) {
  const p = Number(price);
  const d = Number(discountPercent);

  if (!Number.isFinite(p) || !Number.isFinite(d)) {
    throw new TypeError("price and discountPercent must be valid numbers");
  }

  if (p < 0) throw new RangeError("price cannot be negative");
  if (d < 0 || d > 100) throw new RangeError("discountPercent must be
0..100");
  return p - (p * d) / 100;
}
```

Thank you!

Iconography

Our icons are characteristic for Endava's brand appearance and pick up the design principle.

Icons are an essential part of the branding, they can visually express objects, actions and ideas.

Icons can be applied in many different areas. They ease and accelerate the comprehension of functionality and complex content e.g., as functional UI elements in digital applications. Furthermore, icons can also be used together with text to illustrate the content or as oversized illustrative elements.

All icons are constructed on a 24 x 24 px grid. Based on this size icons can be scaled freely but the minimum size should be kept.

See latest version in the Brand Portal

endava